Simulador de Arquitetura Multicore com Gerenciamento de Memória Paginada e Escalonamento Dinâmico de Tarefas

Pedro Henrique Galvão, João Paulo
Departamento de Computação (DECOM-DV)
Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG)
Divinópolis, Brasil
devpedrohg@gmail.com

Igor Peixoto, Otávio Hiratsuka
Departamento de Computação (DECOM-DV)
Centro Federal de Educação Tecnológica de Minas Gerais (CEFET-MG)
Divinópolis, Brasil

Resumo — *[A SER ESCRITO POR OTÁVIO HIRATSUKA] Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Repositório: https://github.com/PeedroHG/multicore-os-simulator.*

Palavras-chave — *Sistemas Operacionais, Multicore, Troca de Contexto, PCB, Lockstep, Amdahl.*

I. Introdução

[Seção sob responsabilidade de Otávio Hiratsuka]

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam.

A. Repositório e Instruções de Compilação

O projeto encontra-se versionado e documentado publicamente no GitHub:

`https://github.com/PeedroHG/multicore-os-simulator`

O fluxo de automação de compilação baseia-se na ferramenta GNU Make e no compilador GCC com suporte ao padrão C++17. A reprodução experimental dos resultados pode ser realizada através dos comandos:

```
# 1. Compilação do executável principal
make

# 2. Execução parametrizada do simulador
./simulador --cores 4 --politica rr --modo p

# 3. Execução da suíte completa de testes
make test

# 4. Execução da bateria de benchmarks
make benchmark
```

II. Arquitetura Multicore e Modelo de Execução

O sistema foi estruturado com um Kernel centralizado, responsável pelo despacho e controle de tempo de n núcleos de processamento idênticos ($\mathrm{CPU}_0, \mathrm{CPU}_1, \ldots, \mathrm{CPU}_{n-1}$). Todos os núcleos compartilham a mesma memória principal, cujo espaço físico é dividido entre a área reservada do sistema operacional (endereços `0x0000` a `0x01FF`) e a área dos programas de usuário.

A. Modelo de Execução Temporal em Lockstep

Para afastar condições de corrida não-determinísticas inerentes à manipulação de threads assíncronas do sistema hospedeiro e assegurar reprodutibilidade experimental exata, a simulação adota o modelo de *passo sincronizado* (*lockstep*).

O Kernel gerencia o relógio central do sistema através do contador `tickGlobal`. Em cada pulso de clock ($t \rightarrow t+1$), o Kernel percorre os núcleos de 0 a $n-1$:

1. Incrementa o tempo de espera (ciclosEspera) de todos os processos que permanecem na fila de aptos (`PRONTO`);
2. Para núcleos em estolagem de memória, consome 1 ciclo de latência de E/S sem despachar instrução;
3. Para núcleos desocupados, aciona o despachante para alocar o próximo processo da fila;
4. Para núcleos com processo ativo, dispara o ciclo elementar de instrução (`CPU::step()`), atualizando registradores e o contador de quantum.

B. Bloco de Controle de Processo (PCB)

A abstração de processo é consolidada na estrutura `PCB` (*Process Control Block*), que atua como o repositório de persistên-

cia do estado lógico e arquitetural de cada tarefa admitida no lote inicial (em $t = 0$). O PCB encapsula:

- **Identificação e Ciclo de Vida:** Identificador numérico exclusivo (`PID`), nome do arquivo fonte e estado formal (`NOVO`, `PRONTO`, `EXECUTANDO`, `FINALIZADO` ou `ABORTADO`);
- **Contexto de Hardware:** Snapshot estruturado (`CPUContext`) composto pelos registradores de uso geral R_0 a R_7, Program Counter (PC) lógico, Instruction Register (IR) e sinalizadores de término da Unidade de Controle (`halted`);
- **Descritores de Escalonamento:** Prioridade estática base, prioridade dinâmica efetiva (para mecanismos de *aging*), estimativa estática de rajada e quantum consumido;
- **Telemetria e Métricas:** *Timestamps* dos ciclos de admissão, início e encerramento, além de contadores para tempo de espera na fila, tempo de execução na CPU, tempo parado em stall e quantidade de trocas de contexto.

C. Mecanismo e Penalidade de Troca de Contexto

A troca de contexto é implementada desacopladamente na classe `CPU` através dos métodos primitivos `salvarContexto(PCB&)` e `restaurarContexto(PCB&)`.

Quando um processo esgota sua fatia temporal (quantum) ou sofre preempção por prioridade, o núcleo extrai os valores dos registradores físicos e os salva no PCB. Em seguida, a CPU carrega os registradores do novo processo que vai assumir o núcleo.

Para refletir o custo operacional real de salvamento de registradores e sobrecarga do despachante, o simulador aplica uma penalidade de tempo configurável:

$$C_{\text{troca}} = 5 \quad \text{ciclos de clock}$$

Essa penalidade é estritamente contabilizada apenas quando há alternância efetiva de processos ($\text{PID}_{\text{anterior}} \neq \text{PID}_{\text{novo}}$) no mesmo núcleo.

D. Tratamento de Faltas de Página via Estolagem (Stall)

Em conformidade com a especificação formal do trabalho que proíbe o chaveamento de tarefas no cenário não-preemptivo, as faltas de bloco/página (*page faults*) geradas pelo subsistema de memória virtual não transitam o processo para um estado tradicional `BLOQUEADO`. Em substituição, a CPU ativa um mecanismo de estolagem do núcleo (*pipeline stall*) por:

$$C_{\text{falta}} = 50 \quad \text{ciclos de clock}$$

somados a 50 ciclos adicionais caso a moldura vítima substituída exija escrita retroativa (*write-back*) de dados alterados para o disco virtual. Durante os ciclos de estolagem, o núcleo permanece paralisado simulando o barramento de transferência de memória secundária, garantindo fidelidade temporal ao cenário não-preemptivo.

E. Fluxograma Geral da Arquitetura

O fluxo operacional do Kernel e a interação síncrona entre núcleos, despachante e memória estão esquematizados na Figura 1.

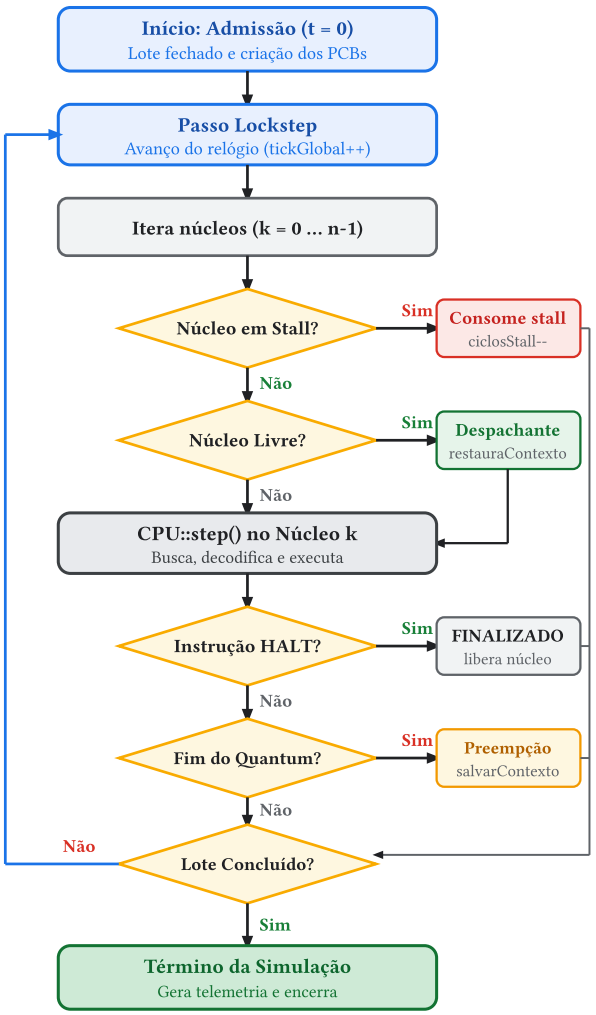


Figura 1: Fluxograma do ciclo de execução multicore em passo sincronizado (Lockstep) gerenciado pelo Kernel.

III. Gerenciamento de Memória Virtual

[Seção sob responsabilidade de João Paulo: Fatiamento em 16 palavras, MMU, Tabela de Molduras, FIFO e LRU]

IV. Políticas de Escalonamento de Tarefas

[Seção sob responsabilidade de Igor Peixoto: FCFS, SJN, RR, Prioridade + Aging e Pseudocódigos Formais]

V. Metodologia Experimental e Cargas de Teste

[Seção sob responsabilidade de Otávio Hiratsuka: Lotes CPU-bound, Memory-bound e Misto]

VI. Resultados e Discussão

A. Análise de Speedup e Escalabilidade Multicore (Experimento E1)

A quantificação do ganho advindo da paralelização física do hardware toma como referência a formulação clássica da Lei de Amdahl. O Speedup $S(n)$ sobre n núcleos de processamento é mensurado comparando o makespan (tempo total de conclusão do lote) obtido na configuração uniprocessada $T(1)$ contra a execução paralela em n núcleos $T(n)$:

$$S(n) = \frac{T(1)}{T(n)}$$

A eficiência paralela normalizada $E(n)$, que traduz a fração de hardware efetivamente convertida em aceleração líquida, é definida por:

$$E(n) = \frac{S(n)}{n} \times 100\%$$

[Os gráficos comparativos e a discussão aprofundada dos resultados serão inseridos após a execução completa dos benchmarks com os lotes finais de teste.]

VII. Conclusão

[Seção sob responsabilidade de Otávio Hiratsuka]

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Referências

1. J. L. Hennessy e D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6ª ed., Morgan Kaufmann, 2017.
2. A. Silberschatz, P. B. Galvin e G. Gagne, *Operating System Concepts*, 10ª ed., Wiley, 2018.
3. A. S. Tanenbaum e H. Bos, *Modern Operating Systems*, 4ª ed., Pearson, 2015.
4. G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities", *AFIPS Conf. Proc.*, 1967.
5. L. A. Belady, "A study of replacement algorithms for a virtual-storage computer", *IBM Systems Journal*, 1966.